

Python Interview Question Bank

50 Core Python Interview Questions — Easy, Medium & Hard, with Full Answers and Follow-Up Notes

What's inside

- **50 conceptual interview questions:** 17 Easy, 17 Medium, 16 Hard
- Covers Python fundamentals, OOP, decorators, generators, concurrency (GIL, threading, multiprocessing, asyncio), memory management, metaclasses, and more
- Every question includes a clear written answer plus a short illustrative code example
- Every answer is followed by **6 key points:** edge cases, common interviewer follow-ups, pitfalls, and related concepts

Table of Contents

Easy (17 questions)

1. List vs Tuple
2. Python Data Types
3. Mutable vs Immutable
4. PEP 8
5. == vs is
6. Indentation and Code Blocks
7. Shallow Copy vs Deep Copy
8. Variable Scope (LEGB)
9. append() vs extend()
10. Dictionaries
11. Syntax Error vs Runtime Error
12. *args and **kwargs
13. Local vs Global Variables
14. Type Casting
15. List Comprehensions
16. remove() vs pop() vs del
17. None in Python

Medium (17 questions)

18. Decorators
19. Generators and yield
20. copy Module Deep Dive
21. The Global Interpreter Lock (GIL)
22. Multithreading vs Multiprocessing
23. Context Managers (with statement)
24. @staticmethod vs @classmethod
25. Duck Typing
26. Magic / Dunder Methods
27. Iterators vs Iterables
28. Monkey Patching
29. try / except / else / finally
30. Lambda Functions
31. Object Equality (__eq__)
32. Namespaces
33. @property vs Getters/Setters
34. Thread Safety Nuances

Hard (16 questions)

35. Memory Management & Garbage Collection
36. Metaclasses
37. Threading vs Multiprocessing vs asyncio
38. Coroutines vs Generators
39. Descriptor Protocol
40. Method Resolution Order (MRO)

41. GIL and CPU-bound vs I/O-bound Threads

42. Weak References

43. `__slots__`

44. Python's Import System

45. pickle vs marshal

46. Closures and Late Binding

47. Process-based vs Thread-based Parallelism for CPU-bound Work

48. Exception Chaining (raise ... from ...)

49. contextvars vs threading.local

50. Small Integer / String Interning and 'is'

Easy Questions

Core Python fundamentals: data types, mutability, scope, and basic language behavior. Good for screening / phone-round interviews.

Q1. List vs Tuple

EASY

Question: What is the difference between a list and a tuple in Python?

Lists are mutable, ordered collections defined with square brackets; tuples are immutable, ordered collections defined with parentheses. Because tuples can't change after creation, they are hashable (usable as dict keys or set members) as long as their contents are also hashable, whereas lists are not hashable and cannot be used as dict keys.

```
my_list = [1, 2, 3]
my_list.append(4)          # OK, lists are mutable

my_tuple = (1, 2, 3)
my_tuple[0] = 99           # TypeError: 'tuple' object does not support item assignment

d = {(1, 2): "point"}      # OK - tuple as dict key
d2 = {[1, 2]: "point"}     # TypeError - list is unhashable
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. A tuple containing a mutable element (e.g. a list) is itself unhashable and can't be used as a dict key.
2. Tuples are generally slightly faster to create and iterate than lists due to their fixed size.
3. Common follow-up: 'why use a tuple at all?' -- to signal intent (fixed-size record) and to enable safe sharing without accidental mutation.
4. Edge case: a single-element tuple requires a trailing comma, e.g. (1,) -- (1) is just an int in parentheses.
5. Pitfall: interviewers often ask to prove tuples are 'immutable' -- note this only applies to the tuple's references, not to mutable objects it contains (my_tuple[0].append(x) can still work if my_tuple[0] is a list).
6. Related: namedtuple and dataclasses(frozen=True) build on tuple-like immutability with named fields.

Q2. Python Data Types

EASY

Question: What are Python's built-in data types?

Python has several built-in categories: numeric types (int, float, complex), sequence types (str, list, tuple, range), mapping type (dict), set types (set, frozenset), boolean (bool, a subclass of int), and NoneType (the type of None). Python is dynamically typed, meaning a variable's type is determined at runtime based on the value assigned to it.

```
print(type(5))          # <class 'int'>
print(type(5.0))        # <class 'float'>
print(type("hi"))       # <class 'str'>
print(type([1, 2]))     # <class 'list'>
print(type((1, 2)))     # <class 'tuple'>
print(type({"a": 1}))   # <class 'dict'>
print(type({1, 2}))     # <class 'set'>
print(type(True))       # <class 'bool'>
print(type(None))       # <class 'NoneType'>
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. bool is technically a subclass of int -- True == 1 and False == 0 both evaluate to True.
2. Dynamic typing means the same variable name can be reassigned to hold different types over its lifetime, unlike statically typed languages.
3. Edge case: dividing two ints with / always returns a float in Python 3 (e.g. 4/2 == 2.0), while // returns an int-like floor division result.
4. Common follow-up: 'is Python strongly or weakly typed?' -- strongly typed (no implicit coercion between unrelated types like '2' + 2 raises TypeError), but dynamically typed.
5. Pitfall: candidates often confuse dynamic typing with weak typing -- Python enforces type rules strictly at operation time even though variables aren't declared with a fixed type.
6. Related: type hints (PEP 484) add optional static-analysis type annotations without changing Python's runtime dynamic typing.

Q3. Mutable vs Immutable

EASY

Question: Explain mutable and immutable objects in Python with examples.

Mutable objects can be changed in place after creation without changing their identity (`id()`) -- examples include list, dict, and set. Immutable objects cannot be changed once created; any 'modification' actually creates a new object -- examples include int, float, str, tuple, and frozenset.

```
s = "hello"
print(id(s))
s += " world"
print(id(s))          # different id -- a new string object was created

lst = [1, 2]
print(id(lst))
lst.append(3)
print(id(lst))        # same id -- modified in place
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Immutability makes objects safe to share across threads and use as dictionary keys, since their hash value never changes.
2. Edge case: strings are immutable, so repeated concatenation in a loop (`s += x`) is $O(n^2)$ overall -- use `".join(parts)"` instead for efficiency.
3. Common follow-up: 'is a tuple always immutable?' -- the tuple's structure is fixed, but if it holds mutable objects (like a list), those nested objects can still change.
4. Pitfall: passing a mutable default argument to a function (`def f(x=[])`) is a classic Python bug -- the same list object persists across calls, silently accumulating state.
5. Related: funtools and default argument evaluation only happens once, at function definition time, not on each call.
6. Interviewers may ask you to demonstrate this with `id()` calls, as shown, to prove whether an operation mutates in place or creates a new object.

Q4. PEP 8

EASY

Question: What is PEP 8 and why does it matter?

PEP 8 is Python's official style guide, covering naming conventions, indentation, line length, whitespace, and import ordering. It matters because consistent style improves readability and maintainability across teams and codebases, and many tools (flake8, black, pylint) enforce or auto-format code according to it.

```
# PEP 8 examples
def calculate_total(price, tax_rate):  # snake_case for functions/variables
    return price * (1 + tax_rate)

class ShoppingCart:                   # PascalCase for classes
    MAX_ITEMS = 50                    # UPPER_CASE for constants

    def __init__(self):
        self._items = []             # leading underscore = "internal use"
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. PEP 8 recommends 4 spaces per indentation level and never mixing tabs and spaces.
2. Naming conventions: snake_case for functions/variables, PascalCase for classes, UPPER_SNAKE_CASE for constants.
3. Common follow-up: 'what tools enforce PEP 8?' -- flake8/pylint for linting, black/autopep8 for auto-formatting.
4. Edge case: PEP 8 is a guideline, not a hard rule -- team style guides can override specific recommendations (e.g. line length) as long as it's applied consistently.
5. Pitfall: conflating PEP 8 (style) with PEP 20 ('The Zen of Python', philosophy) -- interviewers sometimes probe whether you know the difference.
6. Related: type hints (PEP 484) and f-strings (PEP 498) are separate PEPs often discussed alongside PEP 8 in style-related interview questions.

Q5. == vs is

EASY

Question: What is the difference between == and is in Python?

== checks value equality (do these objects have equal content, via `__eq__`), while `is` checks identity equality (are these the exact same object in memory, comparing `id()`). Two variables can hold equal values but be different objects, or in some cases (small int/string caching) be the very same object.

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)  # True (same contents)
print(a is b)  # False (different objects)

c = a
print(a is c)  # True (same object, same id)

x = 256
y = 256
print(x is y)  # True (CPython caches small ints -256..256)

x2 = 257
y2 = 257
print(x2 is y2) # often False (not guaranteed, implementation detail)
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Always use `is` for comparing to `None` (if `x is None`), not `==`, since `is` is faster and avoids relying on a potentially overridden `__eq__`.
2. Edge case: CPython caches small integers (-5 to 256) and some short strings, so `'is'` comparisons on these can misleadingly return `True` -- never rely on this behavior in real code.
3. Common follow-up: 'what does `==` call under the hood?' -- it invokes the object's `__eq__` method, which can be overridden for custom equality logic.
4. Pitfall: using `is` to compare strings or numbers for equality is a common bug -- it works 'by accident' for small cached values but breaks for larger ones or different Python implementations (PyPy, Jython).
5. Edge case: NaN is a special float where `nan == nan` is `False`, but `nan is nan` can be `True` if it's literally the same object.
6. Related: `id()` returns an object's memory address in CPython (implementation detail, not guaranteed by the language spec).

Q6. Indentation and Code Blocks

EASY

Question: Why does Python use indentation instead of braces, and what issues can it cause?

Python uses significant whitespace (indentation) to define code blocks instead of curly braces, enforcing a consistent, readable visual structure across the whole language. The main practical issue is mixing tabs and spaces, or inconsistent indentation levels, which raises an `IndentationError` or `TabError`.

```
def check(x):
    if x > 0:
        print("positive")
    else:
        print("non-positive")

# Mixing tabs and spaces below would raise:
# TabError: inconsistent use of tabs and spaces in indentation
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Python 3 strictly disallows mixing tabs and spaces within the same block (Python 2 allowed it with warnings) -- always configure your editor to use spaces only.
2. Edge case: a completely empty block (like an empty if body) requires the 'pass' statement as a placeholder, since Python can't have a block with zero indented lines.
3. Common follow-up: 'what's the recommended indentation size?' -- PEP 8 recommends 4 spaces per level.
4. Pitfall: copy-pasting code between editors with different tab-width settings is a classic source of hard-to-spot `IndentationErrors`.
5. Edge case: continuation lines (wrapped expressions inside parentheses) don't need to follow the same strict indentation rules as block-defining indentation.
6. Related: this design choice is often cited as a Python philosophy point -- 'readability counts' (from the Zen of Python).

Q7. Shallow Copy vs Deep Copy

EASY

Question: What is the difference between a shallow copy and a deep copy?

A shallow copy creates a new outer object but still references the same nested objects as the original; a deep copy recursively copies all nested objects too, so the copy is fully independent. The copy module provides `copy.copy()` for shallow copies and `copy.deepcopy()` for deep copies.

```
import copy

original = [[1, 2], [3, 4]]

shallow = copy.copy(original)
shallow[0].append(99)
print(original)    # [[1, 2, 99], [3, 4]] -- inner list was shared!

deep = copy.deepcopy(original)
deep[0].append(100)
print(original)    # unaffected, still [[1, 2, 99], [3, 4]]
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Edge case: for a list of immutable elements (e.g. list of ints), shallow copy behaves 'as if' independent since ints can't be mutated in place anyway.
2. Time/space: deepcopy is more expensive (recursively traverses the whole object graph) -- avoid it for large, deeply nested structures unless truly needed.
3. Common follow-up: 'how does list slicing relate to copying?' -- `lst[:]` and `list(lst)` both perform a shallow copy, same as `copy.copy(lst)`.
4. Pitfall: `assuming =` creates a copy at all -- '`new = original`' just creates another reference to the same object, not a copy of any kind.
5. Edge case: deepcopy correctly handles circular references (an object referencing itself) without infinite looping, by tracking already-copied objects internally.
6. Related: custom classes can control deepcopy behavior by implementing `__deepcopy__` and `__copy__` dunder methods.

Q8. Variable Scope (LEGB)

EASY

Question: Explain Python's variable scope resolution order (LEGB).

Python resolves variable names using the LEGB rule: Local scope (inside the current function) is checked first, then Enclosing scope (any outer function, for closures), then Global scope (module-level), and finally Built-in scope (Python's built-in names like `len`, `print`). The first matching name found wins.

```
x = "global"

def outer():
    x = "enclosing"
    def inner():
        x = "local"
        print(x)      # "local"
    inner()
    print(x)          # "enclosing"

outer()
print(x)              # "global"
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The 'global' keyword lets an inner function modify a global-scope variable instead of creating a new local one.
2. The 'nonlocal' keyword lets an inner function modify a variable from its nearest enclosing (non-global) scope -- used in closures.
3. Edge case: referencing a local variable before it's assigned anywhere in the function raises `UnboundLocalError`, even if a global variable of the same name exists -- Python decides scope at compile time based on assignment anywhere in the function body.
4. Common follow-up: 'what happens if you assign to a variable inside a function without 'global'?' -- Python treats it as a new local variable, shadowing the global one.
5. Pitfall: forgetting 'nonlocal' when trying to increment a counter variable in a nested function is a very common closure bug.
6. Related: comprehensions (list/dict/set) have their own local scope in Python 3, unlike Python 2 where list comprehensions leaked their loop variable into the enclosing scope.

Q9. `append()` vs `extend()`

EASY

Question: What is the difference between `list.append()` and `list.extend()`?

`append()` adds a single element to the end of a list (even if that element is itself a list, it gets added as one nested item). `extend()` takes an iterable and adds each of its elements individually to the end of the list, effectively concatenating.

```
lst = [1, 2, 3]
lst.append([4, 5])
print(lst)      # [1, 2, 3, [4, 5]] -- nested list added as one item

lst2 = [1, 2, 3]
lst2.extend([4, 5])
print(lst2)     # [1, 2, 3, 4, 5] -- individual elements added
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Time complexity: `append()` is $O(1)$ amortized; `extend()` is $O(k)$ where k is the length of the iterable being added.
2. Edge case: `extend()` accepts any iterable (string, generator, tuple, set), not just a list -- `extend('ab')` adds 'a' and 'b' as two separate characters.
3. Common follow-up: 'what does `+=` do on a list?' -- `lst += other` behaves like `extend(other)`, not `append(other)`.
4. Pitfall: using `append()` when you meant `extend()` is a classic bug that silently produces a nested list instead of a flat concatenation.
5. Edge case: `extend()` with an empty iterable is a no-op, leaving the list unchanged.
6. Related: `list.insert(index, item)` is a third variant that inserts a single item at a specific position rather than the end.

Q10. Dictionaries

EASY

Question: What is a dictionary in Python and what are its key characteristics?

A dictionary is a built-in mapping type storing key-value pairs, implemented as a hash table for average $O(1)$ lookup, insertion, and deletion. Keys must be hashable (immutable types like str, int, tuple), while values can be any type. Since Python 3.7, dictionaries preserve insertion order as a language guarantee.

```
d = {"name": "Alice", "age": 30}
d["city"] = "NYC"           # insert
print(d.get("age"))         # 30
print(d.get("missing", 0))  # 0 (default, no KeyError)
del d["age"]                # delete
print(list(d.keys()))       # ['name', 'city']
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Time complexity: $O(1)$ average for get/set/delete, $O(n)$ worst case if many hash collisions occur (rare in practice).
2. Edge case: accessing a missing key with `d[key]` raises `KeyError`; use `d.get(key, default)` or `'key' in d` to avoid the exception.
3. Common follow-up: 'can a list be a dict key?' -- no, lists are unhashable/mutable; use a tuple instead if a composite key is needed.
4. Edge case: since Python 3.7, dict maintains insertion order as a guaranteed language feature (not just a CPython implementation detail as it was in 3.6).
5. Pitfall: mutating a dict while iterating over it directly (for `k in d`: `del d[k]`) raises `RuntimeError` -- iterate over a copy (`list(d.keys())`) instead.
6. Related: `collections.defaultdict` and `collections.OrderedDict` provide specialized dict variants for common patterns.

Q11. Syntax Error vs Runtime Error

EASY

Question: What is the difference between a syntax error and a runtime (exception) error?

A `SyntaxError` occurs when Python's parser can't understand the code's grammar (e.g. a missing colon) and is caught before the program even starts running. A runtime error (exception, like `ZeroDivisionError` or `TypeError`) occurs while the program is executing, when a valid-looking statement encounters an invalid operation at runtime.

```
# SyntaxError -- caught at parse time, program never starts
# def foo()
#     return 1

# Runtime error -- caught while running
def divide(a, b):
    return a / b

print(divide(10, 0))  # ZeroDivisionError: division by zero
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Syntax errors prevent the entire module from running, even code that appears before the error in the file.
2. Runtime errors only halt execution at the specific line where they occur; code before it has already run.
3. Common follow-up: 'can you catch a `SyntaxError` with try/except?' -- generally no, since it happens at compile/parse time before try/except logic can run (except when using `exec()` or `eval()` on dynamically-built code strings).
4. Edge case: `IndentationError` and `TabError` are subclasses of `SyntaxError`, specifically for whitespace-related parsing issues.
5. Pitfall: confusing 'logical errors' (code runs without crashing but produces wrong results) with syntax/runtime errors -- logical errors are the hardest to detect since Python raises no exception at all.
6. Related: linters (flake8, pylint) and IDEs catch many syntax errors before you even run the code.

Q12. *args and **kwargs

EASY

Question: What do *args and **kwargs mean in a function signature?

*args collects any extra positional arguments into a tuple, and **kwargs collects any extra keyword arguments into a dictionary. They let a function accept a variable, unspecified number of arguments, and are also used with */** to unpack sequences/dicts when calling a function.

```
def describe(*args, **kwargs):
    print("positional:", args)
    print("keyword:", kwargs)

describe(1, 2, 3, name="Alice", age=30)
# positional: (1, 2, 3)
# keyword: {'name': 'Alice', 'age': 30}

nums = [1, 2, 3]
print(sum(*nums))    # unpacking example (equivalent to sum(nums))
def add(a, b): return a + b
print(add(**{"a": 1, "b": 2})) # 3
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The names 'args' and 'kwargs' are just convention -- any valid identifier works (*values, **options), only the * and ** matter.
2. Edge case: *args must come before **kwargs in a function signature; Python enforces a strict parameter ordering (positional, *args, keyword-only, **kwargs).
3. Common follow-up: 'how do you force keyword-only arguments?' -- define parameters after a bare * in the signature, e.g. `def f(a, *, b):` forces b to be passed by keyword.
4. Edge case: calling a function with **some_dict requires the dict's keys to be strings matching valid parameter names, or it raises TypeError.
5. Pitfall: forgetting that *args is always a tuple (not a list) can cause bugs if code tries to mutate it directly.
6. Related: this pattern is heavily used in decorators and wrapper functions (functools.wraps) to pass arguments through transparently.

Q13. Local vs Global Variables

EASY

Question: What's the difference between local and global variables, and how does Python handle name conflicts?

A local variable is defined inside a function and only accessible within that function's scope; a global variable is defined at module level and accessible throughout the module (and inside functions, for reading). When a local variable shares a name with a global one, the local variable takes precedence inside its function unless explicitly declared global.

```
counter = 0    # global

def increment():
    global counter
    counter += 1

def broken_increment():
    counter += 1    # UnboundLocalError -- treated as local due to assignment

increment()
print(counter)    # 1
# broken_increment()    # would raise UnboundLocalError
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Reading a global variable inside a function works without any special keyword, as long as you don't also assign to it in that function.
2. Edge case: any assignment to a name anywhere within a function body makes Python treat that name as local for the ENTIRE function, even before the assignment line -- causing UnboundLocalError if read first.
3. Common follow-up: 'why avoid global variables?' -- they make code harder to test, reason about, and parallelize since any function can silently mutate shared state.
4. Pitfall: overusing 'global' to work around this is usually a code smell -- passing values as arguments and returning results is generally cleaner.
5. Edge case: global variables inside classes behave differently from instance attributes (self.x) -- don't confuse module-level globals with class-level state.
6. Related: the 'nonlocal' keyword solves the analogous problem for nested functions referring to an enclosing (non-global) scope.

Q14. Type Casting

EASY

Question: What is type casting (type conversion) in Python, and what are its limitations?

Type casting is explicitly converting a value from one type to another using functions like `int()`, `float()`, `str()`, `list()`, etc. Python does NOT perform implicit type coercion between unrelated types (e.g. `'2' + 2` raises `TypeError`), so explicit casting is required when mixing types in operations.

```
print(int("42"))      # 42
print(float("3.14"))  # 3.14
print(str(100))       # "100"
print(int("3.14"))    # ValueError: invalid literal for int()
print(int(3.99))      # 3 (truncates, does not round)
print(list("abc"))    # ['a', 'b', 'c']
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Edge case: `int()` truncates floats toward zero rather than rounding -- `int(3.99)` is 3, not 4; use `round()` if rounding is desired.
2. Edge case: `int('3.14')` raises `ValueError` -- `int()` can't parse a string containing a decimal point directly; convert via `float()` first: `int(float('3.14'))`.
3. Common follow-up: 'does Python auto-convert int to float in arithmetic?' -- yes, in mixed-type numeric operations (`1 + 2.5 == 3.5`), but this is numeric promotion, not casting between unrelated types.
4. Pitfall: casting a non-numeric string like `int('abc')` raises `ValueError` -- always validate or wrap in `try/except` when parsing user input.
5. Edge case: `bool()` casting follows Python's 'truthiness' rules -- `bool(0)`, `bool("")`, `bool([])`, `bool(None)` are all `False`; nearly everything else is `True`.
6. Related: the `ast.literal_eval()` function safely evaluates string literals into Python objects, useful for parsing trusted structured strings.

Q15. List Comprehensions

EASY

Question: What is a list comprehension and why is it preferred over a manual loop for building lists?

A list comprehension is a concise syntax for creating a new list by applying an expression to each item of an iterable, optionally filtered by a condition, all in a single readable line. It's often faster than an equivalent `for`-loop with `.append()` because the looping happens in optimized C code internally.

```
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
pairs = [(x, y) for x in range(3) for y in range(3) if x != y]

# equivalent loop version:
squares2 = []
for x in range(10):
    squares2.append(x**2)
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. List comprehensions create the entire list in memory at once -- for very large datasets, a generator expression (parentheses instead of brackets) is more memory-efficient.
2. Edge case: nested comprehensions (multiple `'for'` clauses) can quickly become unreadable -- prefer a regular loop if there's more than 2 levels of nesting or complex conditionals.
3. Common follow-up: 'what's the difference between a list comprehension and a generator expression?' -- `[x for x in y]` builds a full list eagerly; `(x for x in y)` creates a lazy iterator.
4. Pitfall: putting complex, side-effect-causing logic inside a comprehension (like calling a function that mutates external state) hurts readability -- comprehensions are best for pure transformations.
5. Edge case: dict and set comprehensions use similar syntax: `{k: v for k, v in items}` and `{x for x in items}`.
6. Related: Python 3's comprehensions have their own scope, so the loop variable doesn't leak into the enclosing scope (unlike Python 2).

Q16. remove() vs pop() vs del

EASY

Question: What's the difference between `list.remove()`, `list.pop()`, and the `del` statement?

`remove(value)` deletes the first occurrence of a given VALUE from a list (raises `ValueError` if not found). `pop(index)` removes and returns the element at a given INDEX (defaults to the last element, raises `IndexError` if out of range). `del list[index]` removes an element at an index without returning it, and can also delete a variable/slice entirely.

```
lst = [10, 20, 30, 20]
lst.remove(20)      # removes the FIRST 20 by value
print(lst)          # [10, 30, 20]

popped = lst.pop(0)  # removes and returns index 0
print(popped, lst)   # 10 [30, 20]

del lst[0]           # removes index 0, returns nothing
print(lst)           # [20]

del lst              # deletes the variable entirely
# print(lst)         # NameError: name 'lst' is not defined
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. `remove()` raises `ValueError` if the value isn't present in the list -- wrap in `try/except` or check 'if value in lst' first.
2. `pop()` with no argument removes the LAST element ($O(1)$); `pop(0)` removes the FIRST element but is $O(n)$ since all remaining elements must shift.
3. Common follow-up: 'which is fastest for removing from the end?' -- `pop()` with no index, $O(1)$ amortized.
4. Edge case: `del` can also delete a slice: `del lst[1:3]` removes multiple elements at once.
5. Pitfall: confusing `remove()` (by value) with `pop()` (by index) is one of the most common beginner mistakes -- `remove(2)` removes the VALUE 2, not index 2.
6. Related: for frequent removals from the front of a large collection, `collections.deque` is far more efficient than a list ($O(1)$ vs $O(n)$).

Q17. None in Python

EASY

Question: What is None in Python, and how should it typically be used?

None is Python's singleton representing 'the absence of a value' -- it's the sole instance of `NoneType`. It's commonly used as a default function return value (functions with no explicit return statement return `None`), a default parameter value sentinel, or to signal 'no result found'.

```
def no_return():
    pass

print(no_return())    # None

def find_user(user_id, users):
    return users.get(user_id)    # returns None if not found

result = find_user(99, {1: "Alice"})
if result is None:
    print("User not found")
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Always compare to `None` using 'is `None`' / 'is not `None`', not '`== None`' -- this avoids relying on a custom `__eq__` implementation and is the idiomatic, PEP 8-recommended style.
2. Edge case: `None` is falsy in boolean contexts (`bool(None)` is `False`), but so are `0`, `"`, and `[]` -- 'if `x:`' alone can't distinguish `None` from these other falsy values.
3. Common follow-up: 'why not use `None` as a mutable default argument?' -- actually the opposite pattern is the pitfall: use `None` as the default and create the mutable object inside the function body, e.g. `def f(x=None): x = x or []`.
4. Pitfall: functions that fall off the end without a return statement silently return `None` -- this can cause confusing bugs if the caller expects a different type.
5. Edge case: `None` can be a valid dictionary VALUE (`d = {'key': None}`), which is different from the key being absent -- use `'key in d'` to check presence, not `d.get(key)` is `None`.
6. Related: `Optional[X]` from the typing module documents that a value may be `X` or `None` for static type checkers.

Medium Questions

Language features and design patterns: decorators, generators, context managers, concurrency basics, and object model concepts.

Q18. Decorators

MEDIUM

Question: What are decorators in Python and how do they work?

A decorator is a function that takes another function (or class) as input and returns a modified or wrapped version of it, without changing the original function's source code. The `@decorator` syntax is syntactic sugar for `func = decorator(func)`. They're commonly used for logging, timing, access control, caching, and validation.

```
import functools

def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with {args}, {kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper

@log_calls
def add(a, b):
    return a + b

add(2, 3)
# Calling add with (2, 3), {}
# add returned 5
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. `functools.wraps(func)` is essential -- without it, the wrapped function loses its original `__name__`, `__doc__`, and other metadata, which breaks introspection and debugging.
2. Edge case: decorators with their own arguments (e.g. `@retry(times=3)`) require an extra level of nesting -- a function that returns a decorator, which returns the wrapper.
3. Common follow-up: 'can you stack multiple decorators?' -- yes, they apply bottom-up: `@a \n @b \n def f():` is equivalent to `f = a(b(f))`.
4. Edge case: decorating a method inside a class requires the wrapper to accept 'self' as part of `*args`, which `functools.wraps` handles transparently.
5. Pitfall: decorators that maintain mutable state (like a call counter) in a closure variable are not thread-safe by default -- a lock is needed for concurrent access.
6. Related: class-based decorators (implementing `__call__`) are an alternative to function-based decorators, useful when the decorator needs more complex internal state.

Q19. Generators and yield

MEDIUM

Question: What are generators, and how does the yield keyword differ from return?

A generator is a special type of iterator created by a function containing at least one yield statement. Unlike return, which exits a function immediately and discards its state, yield pauses execution and remembers the function's entire local state, resuming exactly where it left off on the next call to next(). This makes generators memory-efficient for large or infinite sequences since values are produced lazily, one at a time.

```
def count_up_to(n):
    i = 1
    while i <= n:
        yield i
        i += 1

gen = count_up_to(3)
print(next(gen)) # 1
print(next(gen)) # 2
print(next(gen)) # 3
print(next(gen)) # StopIteration raised

for val in count_up_to(3): # generators work directly in for-loops
    print(val)
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Generators compute values lazily/on-demand, so a generator for an infinite sequence (e.g. all natural numbers) never runs out of memory, unlike a list that would need to materialize everything.
2. Edge case: once a generator is exhausted (StopIteration raised), it cannot be restarted -- you must call the generator function again to get a fresh generator object.
3. Common follow-up: 'what's a generator expression?' -- (x**2 for x in range(10)) is the lazy equivalent of a list comprehension, sharing the same efficiency benefits.
4. Edge case: generators can receive values sent into them via gen.send(value), pausing at the yield expression to receive it -- an advanced coroutine-like pattern.
5. Pitfall: trying to get len() of a generator raises TypeError, since generators don't know their total size in advance without fully consuming themselves.
6. Related: yield from delegates to a sub-generator, simplifying generator composition and forming the basis of pre-async/await coroutine patterns.

Question: How does Python's copy module handle circular references and custom objects during deepcopy()?

copy.deepcopy() maintains an internal memo dictionary mapping already-copied object ids to their copies, which prevents infinite recursion on circular references (an object that directly or indirectly refers to itself). For custom classes, deepcopy() uses the object's __deepcopy__ method if defined, otherwise falls back to copying __dict__ recursively via the default mechanism.

```
import copy

class Node:
    def __init__(self, value):
        self.value = value
        self.next = None

a = Node(1)
b = Node(2)
a.next = b
b.next = a # circular reference!

copied_a = copy.deepcopy(a) # does NOT infinite-loop, thanks to the memo dict
print(copied_a.next.next is copied_a) # True -- cycle preserved correctly
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The memo dictionary (id(obj) -> copy) is what makes deepcopy safe for circular references -- without it, naive recursive copying would loop forever.
2. Edge case: deepcopy correctly preserves shared references within the SAME copy operation -- if two attributes point to the same original object, their copies will also point to the same new object (not two separate copies).
3. Common follow-up: 'how do you customize deepcopy for a class?' -- implement __deepcopy__(self, memo) to control exactly what gets copied (e.g. skip copying a database connection attribute).
4. Pitfall: objects holding unpicklable/uncopyable resources (open file handles, database connections, locks) will raise errors on deepcopy unless explicitly excluded via __deepcopy__ or __getstate__/_setstate__.
5. Edge case: copy.copy() (shallow) does NOT use the memo mechanism in the same recursive way since it doesn't recurse into nested objects at all.
6. Related: the same memo-dictionary technique is conceptually similar to how pickle handles object graphs with shared/circular references during serialization.

Q21. The Global Interpreter Lock (GIL)

MEDIUM

Question: What is the GIL, and how does it affect multi-threaded Python programs?

The Global Interpreter Lock is a mutex in CPython that allows only one thread to execute Python bytecode at a time, even on multi-core machines. This means CPU-bound multi-threaded Python code doesn't get true parallel speedup from threads, but I/O-bound code (network calls, file reads, sleep) benefits from threading because the GIL is released during blocking I/O operations.

```
import threading

def cpu_bound_task():
    total = 0
    for i in range(10_000_000):
        total += i
    return total

# Running cpu bound task in multiple threads gives little to no speedup
# because the GIL serializes bytecode execution across threads.

threads = [threading.Thread(target=cpu_bound_task) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
# Roughly the same wall-clock time as running it 4x sequentially (or worse, due to overhead)
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The GIL exists mainly to simplify CPython's memory management (reference counting isn't thread-safe without it).
2. For CPU-bound parallelism, use the multiprocessing module instead of threading -- separate processes each have their own GIL and Python interpreter, achieving true parallelism.
3. Common follow-up: 'does the GIL affect asyncio?' -- asyncio is single-threaded cooperative concurrency, so the GIL isn't a limiting factor there in the same way; it's used for I/O concurrency, not CPU parallelism.
4. Edge case: I/O-bound threading still benefits significantly since the GIL is released during blocking calls like file/network I/O and `time.sleep()`.
5. Pitfall: assuming 'more threads = faster' for CPU-heavy work is a very common misconception among developers moving from other languages.
6. Related: alternative Python implementations like PyPy (STM experiments) or removing the GIL entirely (PEP 703, the 'free-threaded' CPython build) are active areas of ongoing development.

Q22. Multithreading vs Multiprocessing

MEDIUM

Question: When would you choose multiprocessing over multithreading in Python?

Choose multiprocessing for CPU-bound tasks that need true parallel execution across cores, since each process has its own Python interpreter and memory space, bypassing the GIL entirely. Choose multithreading for I/O-bound tasks (network requests, file operations) where the overhead of spawning separate processes and inter-process communication isn't worth it, and threads share memory more cheaply.

```
from multiprocessing import Pool

def square(x):
    return x * x

if __name__ == "__main__":
    with Pool(4) as pool:
        results = pool.map(square, range(10))
    print(results)  # true parallel execution across 4 processes
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Processes have higher memory overhead (each gets its own copy of the interpreter and imported modules) and slower startup than threads.
2. Edge case: sharing data between processes requires special mechanisms (multiprocessing.Queue, Pipe, shared memory via Value/Array) since processes don't share memory by default, unlike threads.
3. Common follow-up: 'what about asyncio instead of both?' -- asyncio is ideal for massively concurrent I/O-bound work (thousands of connections) with lower overhead than one-thread-per-task.
4. Pitfall: objects passed between processes must be picklable -- passing unpicklable objects (like open file handles, lambdas, or database connections) raises errors.
5. Edge case: on Windows, multiprocessing code must be guarded by 'if __name__ == "__main__":' to avoid infinite process spawning, since Windows lacks fork() and re-imports the module in each child process.
6. Related: concurrent.futures provides a unified, higher-level interface (ThreadPoolExecutor / ProcessPoolExecutor) over both threading and multiprocessing.

Q23. Context Managers (with statement)

MEDIUM

Question: What are context managers, and how do you implement a custom one?

A context manager is an object that defines setup and teardown logic to run automatically around a block of code, used via the 'with' statement. It's implemented by defining `__enter__` (setup, runs on entering the block) and `__exit__` (cleanup, runs on exiting, even if an exception occurred). This guarantees resources like files, locks, or connections are properly released regardless of how the block exits.

```
class Timer:
    def __enter__(self):
        import time
        self.start = time.monotonic()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        import time
        elapsed = time.monotonic() - self.start
        print(f"Elapsed: {elapsed:.4f}s")
        return False  # False = don't suppress exceptions

with Timer():
    total = sum(range(1_000_000))

# Equivalent using contextlib for simple cases:
from contextlib import contextmanager
@contextmanager
def timer():
    import time
    start = time.monotonic()
    yield
    print(f"Elapsed: {time.monotonic() - start:.4f}s")
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. `__exit__` receives exception info (`exc_type`, `exc_val`, `exc_tb`) if an exception occurred inside the 'with' block, letting the context manager handle or log it.
2. Edge case: returning True from `__exit__` suppresses the exception (it won't propagate); returning False (or None) lets it propagate normally -- this is a common source of silent-bug bugs if done accidentally.
3. Common follow-up: 'what's the simplest way to write a context manager?' -- the `@contextlib.contextmanager` decorator turns a generator function (with a single yield) into a context manager without writing a full class.
4. Pitfall: forgetting to release a resource in `__exit__` (e.g. not calling `file.close()` or `lock.release()`) defeats the entire purpose of using a context manager.
5. Edge case: 'with' statements support multiple context managers on one line: `with open('a') as a, open('b') as b:` -- both are properly closed even if one setup fails partway through.
6. Related: this is the same pattern behind 'with open(path) as f:' for files and 'with lock:' for `threading.Lock`, both extremely common in real codebases.

Q24. @staticmethod vs @classmethod

MEDIUM

Question: What is the difference between @staticmethod and @classmethod?

A @classmethod receives the class itself as its first argument (conventionally named cls), letting it access or modify class-level state and be used as an alternative constructor. A @staticmethod receives neither an instance (self) nor the class (cls) -- it behaves like a plain function that's simply namespaced inside the class for organizational purposes.

```
class Pizza:
    def __init__(self, toppings):
        self.toppings = toppings

    @classmethod
    def margherita(cls):
        return cls(["mozzarella", "tomato"]) # alternative constructor

    @staticmethod
    def validate_topping(name):
        return isinstance(name, str) and len(name) > 0

p = Pizza.margherita()
print(p.toppings) # ['mozzarella', 'tomato']
print(Pizza.validate_topping("basil")) # True
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. classmethods are commonly used as alternative/named constructors (like `Pizza.margherita()`) since `cls` lets you correctly instantiate subclasses too, unlike a hardcoded `Pizza(...)` call.
2. Edge case: calling a classmethod on a subclass automatically passes the subclass as `cls`, not the parent class -- this enables proper polymorphic construction.
3. Common follow-up: 'why not just use a regular function outside the class for a staticmethod?' -- staticmethods group related utility logic with the class namespace for organization and discoverability, even without needing `self/cls`.
4. Pitfall: trying to access instance attributes (`self.x`) inside a staticmethod is impossible since it has no access to `self` at all -- if you need instance data, it should be a regular instance method.
5. Edge case: both classmethod and staticmethod can be called on an instance OR the class directly (`Pizza.margherita()` and `p.margherita()` both work identically).
6. Related: `abc.abstractmethod` can be combined with these decorators to define abstract class-level or static interfaces in abstract base classes.

Q25. Duck Typing

MEDIUM

Question: What is duck typing in Python?

Duck typing is a programming style where an object's suitability for an operation is determined by whether it has the required methods/attributes, not by its explicit type or class hierarchy. The idea comes from 'if it walks like a duck and quacks like a duck, it's a duck' -- Python doesn't check types before calling methods, it just tries and either succeeds or raises `AttributeError/TypeError`.

```
class Duck:
    def quack(self):
        return "Quack!"

class Person:
    def quack(self):
        return "I'm quacking like a duck!"

def make_it_quack(thing):
    return thing.quack() # no type check -- just calls .quack()

print(make_it_quack(Duck())) # Quack!
print(make_it_quack(Person())) # I'm quacking like a duck!
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. This is the foundation of Python's 'protocols' -- e.g. any object with `__len__` works with `len()`, any object with `__iter__` works in a for-loop, regardless of its class.
2. Edge case: duck typing means errors surface later (at call time) rather than earlier (at compile/type-check time) -- a tradeoff for flexibility over early error detection.
3. Common follow-up: 'how does duck typing relate to abstract base classes (ABCs)?' -- ABCs (via `collections.abc`) let you formally register or check for expected interfaces while keeping duck-typing flexibility, often via `isinstance()` checks against a Protocol.
4. Pitfall: relying purely on duck typing without any validation can cause confusing `AttributeErrors` deep inside library code when an incompatible object is passed in.
5. Edge case: `typing.Protocol` (PEP 544, 'structural subtyping') brings static-analysis support for duck typing, letting type checkers verify duck-typed interfaces without requiring explicit inheritance.
6. Related: EAFP ('Easier to Ask Forgiveness than Permission') is Python's related idiom -- try the operation and catch exceptions, rather than checking types/conditions upfront (LBYL, 'Look Before You Leap').

Question: What are dunder (magic) methods, and give a few important examples.

Dunder methods (double-underscore methods like `__init__`, `__str__`, `__eq__`) let custom classes integrate with Python's built-in syntax and functions -- operator overloading, iteration, context managers, string representation, and more. Python calls them implicitly in response to built-in syntax (e.g. `len(obj)` calls `obj.__len__()`).

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __repr__(self):
        return f"Vector({self.x}, {self.y})"

    def __eq__(self, other):
        return self.x == other.x and self.y == other.y

    def __len__(self):
        return int((self.x**2 + self.y**2) ** 0.5)

v1, v2 = Vector(1, 2), Vector(3, 4)
print(v1 + v2)    # Vector(4, 6) -- uses __add__
print(v1 == Vector(1, 2)) # True -- uses __eq__
print(len(v2))    # 5 -- uses __len__
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. `__repr__` should return an unambiguous, ideally eval-able string for debugging; `__str__` should return a human-readable string -- if only `__repr__` is defined, it's used as a fallback for both.
2. Edge case: implementing `__eq__` without also implementing `__hash__` makes instances unhashable by default (Python sets `__hash__` to `None` automatically) -- define both if the object needs to go in a `set`/`dict`.
3. Common follow-up: 'what's the difference between `__new__` and `__init__`?' -- `__new__` actually creates the instance (rarely overridden except for immutable types or metaclasses), while `__init__` initializes an already-created instance.
4. Pitfall: defining `__eq__` inconsistently with `__hash__` (equal objects must have equal hashes) breaks dictionary/set behavior in subtle, hard-to-debug ways.
5. Edge case: `__getitem__` and `__setitem__` let a custom class support bracket indexing (`obj[key]`), even implementing custom slicing behavior.
6. Related: `__enter__`/`__exit__` (context managers), `__iter__`/`__next__` (iterators), and `__call__` (making instances callable like functions) are other very commonly asked-about dunder methods.

Q27. Iterators vs Iterables

MEDIUM

Question: What is the difference between an iterable and an iterator?

An iterable is any object that implements `__iter__()` and can be looped over (lists, tuples, dicts, strings) -- calling `iter()` on it returns an iterator. An iterator is an object that implements both `__iter__()` (returning itself) and `__next__()` (returning the next value, raising `StopIteration` when exhausted) -- it maintains state about where it currently is in the sequence.

```
nums = [1, 2, 3]          # iterable (has __iter__)
it = iter(nums)           # iterator (has __iter__ AND __next__)

print(next(it))           # 1
print(next(it))           # 2
print(next(it))           # 3
# print(next(it))         # StopIteration raised

# A list can be iterated multiple times (fresh iterator each time):
for x in nums: pass
for x in nums: pass       # works again, no problem

# But an iterator is exhausted after one full pass:
it2 = iter(nums)
for x in it2: pass
for x in it2: pass        # does nothing -- it2 is already exhausted
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Every iterator is also an iterable (since it implements `__iter__`, typically returning self), but not every iterable is an iterator (a list has `__iter__` but no `__next__`).
2. Edge case: a for-loop implicitly calls `iter()` on its target once at the start, then repeatedly calls `next()` -- this is why iterables can be looped multiple times but the SAME iterator object cannot.
3. Common follow-up: 'how do you make a custom class iterable?' -- implement `__iter__` (returning an iterator, often self if `__next__` is also implemented) and `__next__` (raising `StopIteration` when done).
4. Pitfall: trying to reuse an exhausted iterator (like a generator that's already been fully consumed) silently produces no more values instead of raising an obvious error -- a common source of 'why is my loop not running' bugs.
5. Edge case: generators are a special, convenient case of iterators created via `yield`, automatically implementing both `__iter__` and `__next__`.
6. Related: `itertools` module functions (`islice`, `chain`, `cycle`, `tee`) operate on and produce iterators for memory-efficient lazy processing.

Q28. Monkey Patching

MEDIUM

Question: What is monkey patching in Python, and what are its risks?

Monkey patching is dynamically modifying or extending a class or module at runtime -- for example, replacing a method on an existing class after it's already defined, often used for testing (mocking) or patching bugs in third-party libraries without editing their source. It's powerful but risky because it can silently change behavior everywhere that class/module is used, making bugs hard to trace.

```
class Greeter:
    def greet(self):
        return "Hello!"

def new_greet(self):
    return "Hi there!"

Greeter.greet = new_greet  # monkey-patch the method
g = Greeter()
print(g.greet())  # "Hi there!"

# Common legitimate use: mocking in tests
import unittest.mock as mock
with mock.patch("os.path.exists", return_value=True):
    import os
    print(os.path.exists("/nonexistent/path"))  # True (patched)
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Monkey patching a widely-used library method affects ALL code using that class, including code you don't own or control -- it's a global, not local, change.
2. Edge case: monkey-patched changes typically don't persist across process restarts (unless written to a config/plugin loaded at startup) -- it's a runtime-only modification.
3. Common follow-up: 'what's a safer alternative to monkey patching?' -- subclassing and overriding the method, or using dependency injection, are generally preferred for maintainability.
4. Pitfall: patching a method without restoring the original afterward (especially in tests) can cause 'test pollution' where one test's patch leaks into and breaks subsequent tests.
5. Edge case: `unittest.mock.patch` used as a context manager or decorator automatically restores the original behavior after the block/test completes, avoiding the pollution pitfall.
6. Related: this technique is core to how many testing/mocking libraries work, but is generally discouraged in production application code outside of well-scoped testing utilities.

Q29. try / except / else / finally

MEDIUM

Question: Explain the roles of try, except, else, and finally in Python's exception handling.

'try' contains code that might raise an exception. 'except' catches and handles specific exception types if one is raised. 'else' runs only if the try block completed WITHOUT any exception. 'finally' always runs, regardless of whether an exception occurred, was caught, or even if a return statement executed inside try/except -- typically used for guaranteed cleanup.

```
def read_number(s):
    try:
        value = int(s)
    except ValueError:
        print("Not a valid number")
        return None
    else:
        print("Parsing succeeded")
        return value
    finally:
        print("Done attempting to parse")

read_number("42")    # Parsing succeeded / Done attempting to parse -> 42
read_number("abc")  # Not a valid number / Done attempting to parse -> None
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. 'else' is often overlooked but useful -- it clearly separates 'code that might fail' (try) from 'code that should only run on success' (else), improving readability over cramming everything into try.
2. Edge case: 'finally' runs even if the except block itself raises a new exception, or if try/except contains a return/break/continue -- it's the one guaranteed-to-execute block.
3. Common follow-up: 'what happens if finally also has a return statement?' -- it silently overrides/swallows any exception or return value from try/except, which is usually an unintended bug.
4. Pitfall: catching a bare 'except:' (no exception type specified) catches EVERYTHING including KeyboardInterrupt and SystemExit, which usually should be allowed to propagate -- always catch specific exception types.
5. Edge case: multiple except clauses are checked top-to-bottom, and only the FIRST matching one runs -- order matters, put more specific exceptions before more general ones (e.g. before a bare Exception).
6. Related: 'raise ... from ...' explicitly chains exceptions to preserve the original traceback context when re-raising a different exception type.

Q30. Lambda Functions

MEDIUM

Question: What are lambda functions, and when should (or shouldn't) you use them?

A lambda is a small, anonymous, single-expression function defined inline with the lambda keyword, commonly used as a short callback passed to functions like `sorted()`, `map()`, or `filter()`. Lambdas are restricted to a single expression (no statements, no multiple lines), and PEP 8 discourages assigning a lambda to a variable name -- use a regular 'def' function instead in that case.

```
nums = [(1, 'b'), (2, 'a'), (3, 'c')]
sorted_by_second = sorted(nums, key=lambda pair: pair[1])
print(sorted_by_second)  # [(2, 'a'), (1, 'b'), (3, 'c')]

squared = list(map(lambda x: x**2, [1, 2, 3]))
print(squared)           # [1, 4, 9]

# Discouraged (PEP 8): assigning lambda to a name
# square = lambda x: x**2      # bad -- use 'def square(x): return x**2' instead
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Lambdas can't contain statements (assignments, loops, multi-line logic, try/except) -- only a single expression, which limits their use to simple transformations.
2. Edge case: lambdas capture variables by reference (late binding), not by value -- a classic bug arises when creating lambdas in a loop, e.g. `[lambda: i for i in range(3)]` all return 2, since 'i' is looked up at call time, not creation time.
3. Common follow-up: 'how do you fix the late-binding loop bug?' -- use a default argument to capture the value at creation time: `lambda i=i: i`.
4. Pitfall: overusing lambdas for anything beyond trivial one-liners hurts readability and debuggability (no meaningful `__name__`, harder to add docstrings/type hints) -- a named function is usually clearer.
5. Edge case: lambdas can still form closures over enclosing-scope variables just like regular nested functions.
6. Related: operator module functions (`operator.itemgetter`, `operator.attrgetter`) are often faster and more readable alternatives to simple lambdas used as sort/filter keys.

Q31. Object Equality (__eq__)

MEDIUM

Question: How does Python determine if two custom objects are 'equal', and what must you be careful about when overriding __eq__?

By default, __eq__ (inherited from object) compares identity (same as 'is') -- two distinct instances are never equal unless you override __eq__ to compare their meaningful attributes. When you override __eq__, you should also override __hash__ consistently, since Python's rule is that equal objects MUST have equal hash values (otherwise dict/set behavior breaks).

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __eq__(self, other):
        if not isinstance(other, Point):
            return NotImplemented
        return self.x == other.x and self.y == other.y

    def __hash__(self):
        return hash((self.x, self.y))

p1, p2 = Point(1, 2), Point(1, 2)
print(p1 == p2)          # True (value equality via __eq__)
print(p1 is p2)          # False (different objects)
print({p1, p2})          # a set of size 1, since they hash equal and compare equal
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Returning NotImplemented (not False) from __eq__ when comparing against an incompatible type lets Python correctly fall back to the other object's __eq__ or default to False, supporting proper two-way comparison.
2. Edge case: defining __eq__ without __hash__ makes the class unhashable (Python sets __hash__ = None automatically) -- this silently breaks usage in sets/dicts with a TypeError only at that point of use.
3. Common follow-up: 'what does the equal-objects-must-have-equal-hashes rule protect against?' -- it ensures dict/set lookups work correctly, since these structures use hash values to locate a bucket before confirming equality.
4. Pitfall: making a mutable object's __hash__ depend on mutable fields (like a changing list attribute) is dangerous -- if the object changes after being added to a set, it becomes 'lost' since its hash bucket no longer matches.
5. Edge case: dataclasses (@dataclass) auto-generate a sensible __eq__ (and optionally __hash__ with frozen=True) based on declared fields, reducing this boilerplate.
6. Related: __lt__, __le__, __gt__, __ge__ (or functools.total_ordering) similarly need careful, consistent implementation to support sorting custom objects correctly.

Q32. Namespaces

MEDIUM

Question: What is a namespace in Python, and how does Python organize different namespaces?

A namespace is a mapping from names (identifiers) to objects, essentially a dictionary that Python consults during name resolution. Python maintains several separate namespaces simultaneously: built-in (Python's core functions/types), global/module-level, and local (inside each function call) -- this is exactly what the LEGB rule navigates when resolving a name.

```
x = 10          # lives in the module's global namespace

def outer():
    y = 20      # lives in outer's local namespace
    def inner():
        z = 30  # lives in inner's local namespace
        print(x, y, z)  # resolves x globally, y from enclosing, z locally
    inner()

outer()

print(globals().keys())  # inspect the global namespace directly
print(locals())          # inspect the current local namespace
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Each function call creates a brand-new local namespace -- this is why recursive calls don't clash: each recursive invocation has its own independent set of local variables.
2. Edge case: class bodies create their own namespace during class definition, which becomes the class's `__dict__` -- methods and class attributes live there, separate from any instance's `__dict__`.
3. Common follow-up: 'what's the difference between `globals()` and `locals()`?' -- `globals()` returns the actual, mutable module-level namespace dict (modifying it affects real globals); `locals()` inside a function returns a SNAPSHOT copy in CPython, so modifying it doesn't reliably affect actual local variables.
4. Pitfall: relying on modifying the dict returned by `locals()` to change local variables is unreliable/implementation-specific behavior -- don't do this in real code.
5. Edge case: 'import module' creates a name in the current namespace pointing to the module object; 'from module import name' instead copies a specific name directly into the current namespace.
6. Related: this whole system is why Python needs the 'global' and 'nonlocal' keywords -- they explicitly tell Python which namespace an assignment should target.

Q33. @property vs Getters/Setters

MEDIUM

Question: How does Python's @property decorator compare to traditional getter/setter methods (as in Java)?

@property lets you define methods that are accessed with plain attribute syntax (obj.value) while still running custom logic (validation, computed values, lazy evaluation) behind the scenes -- unlike Java-style explicit get_value()/set_value() calls. This keeps the public API clean and Pythonic while still allowing encapsulation to be added later without breaking existing code that already uses simple attribute access.

```
class Temperature:
    def __init__(self, celsius=0):
        self._celsius = celsius

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self):          # read-only computed property
        return self._celsius * 9/5 + 32

t = Temperature(25)
print(t.fahrenheit)              # 77.0 -- looks like an attribute, computed on access
t.celsius = -300                  # raises ValueError
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. A key Python idiom: start with plain public attributes (self.celsius = value); only add @property later IF validation/computation becomes necessary -- this doesn't break the class's external interface.
2. Edge case: a property without a corresponding @x.setter is read-only -- attempting t.fahrenheit = 100 raises AttributeError since there's no setter defined.
3. Common follow-up: 'why not just use get_x()/set_x() methods like other languages?' -- @property achieves the same encapsulation benefits while preserving clean, attribute-style syntax at call sites.
4. Pitfall: putting expensive or side-effect-heavy logic inside a property getter is surprising to callers, since obj.value LOOKS like a cheap attribute access but might actually do significant work.
5. Edge case: properties can also define a @x.deleter to customize behavior for 'del obj.x'.
6. Related: functools.cached_property is a related decorator that computes a property once and caches the result on the instance, useful for expensive derived values that don't change.

Question: Given the GIL, are Python's built-in data structures like list and dict automatically thread-safe?

Individual, atomic bytecode operations (like `list.append()` or `dict[key] = value`) are effectively thread-safe due to the GIL, since only one thread executes bytecode at a time. However, compound operations made of MULTIPLE steps (like 'if key not in d: `d[key] = value`', or '`x += 1`') are NOT atomic as a whole and can still race between threads, since the GIL can switch threads between the individual bytecode instructions that make up the compound operation.

```
import threading

counter = {"count": 0}

def increment():
    for _ in range(100_000):
        counter["count"] += 1    # NOT atomic: read, add, write are 3 separate steps

threads = [threading.Thread(target=increment) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()

print(counter["count"])    # often LESS than 400,000 due to race conditions

# Fix with a lock:
lock = threading.Lock()
def safe_increment():
    for _ in range(100_000):
        with lock:
            counter["count"] += 1
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The GIL guarantees atomicity only for SINGLE bytecode instructions, not for logical operations spanning multiple bytecode instructions like `+=`, which decomposes into LOAD, ADD, STORE.
2. Edge case: simple single-step operations like `list.append(x)` or `d[k] = v` ARE safe from corruption/crashes because they map to a single (or GIL-protected) bytecode-level operation, though the ORDER of interleaved operations across threads is still unpredictable.
3. Common follow-up: 'so do I still need locks in Python threading code?' -- yes, absolutely, for any compound read-modify-write sequence, exactly as in any other language.
4. Pitfall: assuming 'Python has a GIL, so my threaded code is automatically safe' is a very common and dangerous misconception that leads to subtle race-condition bugs.
5. Edge case: `queue.Queue` is specifically designed to be fully thread-safe for producer/consumer patterns, handling all necessary locking internally so you don't have to.
6. Related: `threading.Lock`, `RLock` (reentrant), and `Semaphore` are the standard tools for protecting compound operations against race conditions in threaded code.

Hard Questions

Advanced internals: memory management, metaclasses, descriptors, concurrency models in depth, and CPython implementation details. Suited for senior-level interviews.

Q35. Memory Management & Garbage Collection

HARD

Question: Explain how Python manages memory, including reference counting and cyclic garbage collection.

CPython primarily uses reference counting: every object tracks how many references point to it, and when that count hits zero, the object is immediately deallocated. This alone can't handle reference CYCLES (e.g. two objects referencing each other but unreachable from anywhere else), so CPython also runs a generational cycle-detecting garbage collector (the 'gc' module) periodically to find and clean up these unreachable cycles.

```
import sys, gc

a = []
print(sys.getrefcount(a)) # includes the temporary ref from getrefcount's own argument

class Node:
    def __init__(self):
        self.ref = None

x, y = Node(), Node()
x.ref = y
y.ref = x # circular reference -- refcount alone can't free x or y

del x, y
gc.collect() # the generational GC finds and frees the unreachable cycle

print(gc.get_count()) # (gen0, gen1, gen2) object counts since last collection
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Reference counting overhead means every assignment/deletion involves incrementing/decrementing counters -- this is a big part of why the GIL exists, to keep these counter updates thread-safe without heavier locking.
2. The generational GC only needs to run for objects that create cycles -- most simple objects (ints, strings, non-cyclic containers) are cleaned up instantly via refcounting alone, without ever touching the cycle collector.
3. Common follow-up: 'what are the 3 generations in Python's GC?' -- new objects start in generation 0; objects that survive a collection get promoted to generation 1, then 2 -- older generations are scanned less frequently since long-lived objects are statistically less likely to become garbage.
4. Edge case: objects with a `__del__` method involved in reference cycles used to be uncollectable in old Python versions (pre-3.4) -- modern CPython (3.4+) can safely collect these too via PEP 442.
5. Pitfall: manually calling `gc.disable()` to 'optimize' performance can cause severe memory bloat in long-running programs with cyclic data structures (e.g. certain tree/graph structures, or frameworks with circular parent-child references).
6. Related: `weakref` lets you hold a reference to an object WITHOUT increasing its reference count, useful for caches or observer patterns to avoid creating unwanted reference cycles in the first place.

Question: What is a metaclass in Python, and when might you use one?

A metaclass is 'the class of a class' -- it controls how classes themselves are created, just as a class controls how its instances are created. By default, all classes are instances of the built-in 'type' metaclass; you can create a custom metaclass (by subclassing type) to customize class creation, such as auto-registering subclasses, enforcing coding conventions, or injecting methods/attributes into every class that uses it.

```
class Meta(type):
    def __new__(mcs, name, bases, namespace):
        # Enforce that every method name is lowercase
        for key in namespace:
            if callable(namespace[key]) and not key.startswith("__"):
                if key != key.lower():
                    raise TypeError(f"Method '{key}' must be lowercase")
        return super().__new__(mcs, name, bases, namespace)

class MyClass(metaclass=Meta):
    def valid_method(self):
        pass

# class BadClass(metaclass=Meta):
#     def BadMethod(self):      # raises TypeError at class-definition time!
#         pass
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Metaclasses intercept CLASS creation (when 'class Foo:' is executed), not instance creation -- `__new__`/`__init__` on a metaclass run once per class defined, not once per instance.
2. Edge case: multiple inheritance with classes using DIFFERENT, incompatible metaclasses raises 'TypeError: metaclass conflict' -- Python requires a consistent metaclass hierarchy.
3. Common follow-up: 'when should you actually use a metaclass?' -- rarely; the Python community mantra is 'metaclasses are deeper magic than 99% of users should worry about' -- class decorators or `__init_subclass__` often achieve similar goals more simply.
4. Pitfall: overusing metaclasses for things achievable with simpler tools (class decorators, `__init_subclass__` hooks) makes code significantly harder to understand and debug.
5. Edge case: Django's ORM and SQLAlchemy's declarative base are real-world examples that use metaclasses to auto-register model classes and wire up class-level field descriptors.
6. Related: `__init_subclass__` (added in Python 3.6) is a simpler, more accessible alternative to metaclasses for many subclass-customization use cases, without needing a separate metaclass definition.

Q37. Threading vs Multiprocessing vs asyncio

HARD

Question: Compare threading, multiprocessing, and asyncio as Python's three concurrency models.

Threading provides concurrent execution within one process, good for I/O-bound work, limited by the GIL for CPU-bound work. Multiprocessing runs separate OS processes with independent memory and their own GIL each, achieving true CPU parallelism at the cost of higher memory/IPC overhead. asyncio provides single-threaded cooperative concurrency using an event loop and coroutines (async/await), ideal for handling thousands of concurrent I/O-bound operations (like network connections) with very low overhead per task.

```
import asyncio

async def fetch(name, delay):
    print(f"{name} starting")
    await asyncio.sleep(delay) # yields control back to the event loop
    print(f"{name} done")
    return f"{name} result"

async def main():
    results = await asyncio.gather(
        fetch("A", 2), fetch("B", 1), fetch("C", 3)
    )
    print(results)

asyncio.run(main())
# All three "start" immediately, interleaved; total time ~3s, not 2+1+3=6s
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. asyncio requires cooperative multitasking -- a task must explicitly 'await' something to yield control; a single CPU-bound, blocking call inside an async function freezes the ENTIRE event loop, unlike threads which are preempted by the OS.
2. Edge case: mixing blocking (synchronous, non-async) library calls inside async code without care (e.g. calling requests.get instead of an async HTTP client) defeats asyncio's concurrency benefits entirely.
3. Common follow-up: 'can you combine these models?' -- yes, e.g. asyncio.to_thread() or loop.run_in_executor() offload blocking/CPU-bound work to a thread or process pool from within async code.
4. Pitfall: choosing multiprocessing for I/O-bound work (like many simultaneous network requests) wastes memory and startup overhead compared to asyncio or threading, which are much more lightweight for that use case.
5. Edge case: asyncio.gather() runs coroutines concurrently and by default propagates the FIRST exception raised, potentially cancelling other still-running tasks -- return_exceptions=True changes this behavior to collect exceptions instead of raising.
6. Related: the general rule of thumb: CPU-bound -> multiprocessing; I/O-bound with few connections -> threading; I/O-bound with MANY concurrent connections -> asyncio.

Q38. Coroutines vs Generators

HARD

Question: How do async/await coroutines relate to generators under the hood?

Historically (pre-3.5), Python coroutines were literally implemented using generators with `yield` used to pause/resume execution and `yield from` to delegate between them. Modern Python (3.5+) has native coroutines defined with `'async def'` and `'await'`, which are conceptually similar (pausable, resumable functions) but are a distinct type from generators at the language level, with cleaner syntax and better static-analysis support.

```
# Old-style (generator-based) coroutine, Python < 3.5 pattern:
import types
@types.coroutine
def old_style_sleep(seconds):
    yield ('sleep', seconds)

# Modern native coroutine:
async def modern_sleep(seconds):
    await asyncio.sleep(seconds)

async def main():
    await modern_sleep(1)
    print("done")

import asyncio
asyncio.run(main())
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Native coroutines (`async def`) cannot be iterated with a plain `for`-loop or `next()` the way generators can -- they must be driven by an event loop via `await` or `asyncio.run()`.
2. Edge case: calling an `async` function directly (`my_coroutine()`) does NOT execute its body -- it returns a coroutine OBJECT that must be awaited or scheduled on an event loop to actually run.
3. Common follow-up: 'what's an `async` generator?' -- a function with both `'async def'` and `'yield'` (PEP 525), producing an object usable in `'async for'` loops, combining lazy iteration with `async/await` pausing.
4. Pitfall: forgetting `'await'` on a coroutine call is a very common bug -- Python typically issues a `'RuntimeWarning: coroutine was never awaited'` rather than an outright error, so the mistake can go unnoticed.
5. Edge case: `'yield from'` (generator delegation) and `'await'` (coroutine delegation) are conceptually parallel but not interchangeable across native coroutine and generator syntax.
6. Related: understanding this history explains many quirks in older `asyncio`-adjacent codebases that still use `@asyncio.coroutine`-style generator syntax, which was fully removed in Python 3.11.

Q39. Descriptor Protocol

HARD

Question: What is the descriptor protocol, and how does it power features like @property?

A descriptor is any object implementing `__get__`, `__set__`, and/or `__delete__`, placed as a CLASS attribute, that customizes attribute access on INSTANCES of that class. When you access `instance.attr`, Python checks if the class has a descriptor for 'attr' and, if so, calls its `__get__` method instead of doing a simple dictionary lookup -- this is exactly how `@property`, methods, and even class-level type-checking attributes are implemented internally.

```
class PositiveNumber:
    def __set_name__(self, owner, name):
        self.name = "_" + name

    def __get__(self, instance, owner):
        if instance is None:
            return self
        return getattr(instance, self.name, 0)

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError(f"{self.name} must be non-negative")
        setattr(instance, self.name, value)

class Account:
    balance = PositiveNumber() # descriptor as a class attribute

    def __init__(self, balance):
        self.balance = balance # triggers __set__

a = Account(100)
print(a.balance) # 100, triggers __get__
a.balance = -50 # raises ValueError
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Descriptors are 'data descriptors' if they define both `__get__` and `__set__` (or `__delete__`) -- these take priority over an instance's own `__dict__` during attribute lookup; 'non-data descriptors' (only `__get__`) can be overridden by instance attributes.
2. Edge case: functions themselves are non-data descriptors -- this is precisely the mechanism that turns a plain function defined in a class body into a bound method when accessed via an instance.
3. Common follow-up: 'how does this relate to @property?' -- property is implemented as a built-in data descriptor class; the `@property` decorator syntax is just a convenient way to create one.
4. Pitfall: forgetting `__set_name__` (or hardcoding the storage attribute name) can cause ALL instances sharing the same descriptor class to collide if the descriptor instance itself is used to store state directly rather than delegating to the owning instance's `__dict__`.
5. Edge case: the `instance=None` case in `__get__` handles ACCESSING the descriptor via the class itself (`Account.balance`) rather than an instance, which should typically return the descriptor object itself for introspection.
6. Related: this same protocol underlies ORMs' field definitions (Django/SQLAlchemy model fields), where each field type is a descriptor controlling validation and storage on model instances.

Q40. Method Resolution Order (MRO)

HARD

Question: What is Method Resolution Order, and how does Python resolve method calls in multiple inheritance?

MRO is the specific, deterministic order in which Python searches parent classes for a method or attribute when using multiple inheritance. CPython uses the C3 linearization algorithm, which produces a consistent order respecting both each class's local base-class ordering and a global 'always check a class before its parents' constraint, avoiding the ambiguity that simpler depth-first search would create (the classic 'diamond problem').

```
class A:
    def greet(self):
        return "A"

class B(A):
    def greet(self):
        return "B -> " + super().greet()

class C(A):
    def greet(self):
        return "C -> " + super().greet()

class D(B, C):
    def greet(self):
        return "D -> " + super().greet()

print(D().greet())          # D -> B -> C -> A
print(D.__mro__)            # (D, B, C, A, object) -- the resolution order
print([c.__name__ for c in D.mro()])
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. `super()` doesn't necessarily call the immediate PARENT class -- it calls the NEXT class in the MRO chain, which depends on the full inheritance graph, not just the current class's direct bases.
2. Edge case: the diamond problem (D inherits B and C, both inherit A) is resolved so `A.greet()` is only called ONCE in the chain via `super()`, not once per path, avoiding duplicate execution.
3. Common follow-up: 'when does Python raise a `TypeError` for MRO?' -- when the C3 algorithm can't produce a consistent linearization (contradictory base class orderings across different classes) -- Python raises 'Cannot create a consistent MRO'.
4. Pitfall: assuming `super().method()` calls a specific known class is a common source of bugs in complex multiple-inheritance hierarchies -- always inspect `ClassName.__mro__` when the actual call order isn't obvious.
5. Edge case: `object` is always the last class in any MRO chain, since every class implicitly inherits from it.
6. Related: mixin classes are a common, more disciplined use of multiple inheritance that relies heavily on understanding MRO and cooperative `super()` calls to compose behavior cleanly.

Q41. GIL and CPU-bound vs I/O-bound Threads

HARD

Question: In detail, why does the GIL hurt CPU-bound threaded performance but not I/O-bound threaded performance?

The GIL is released periodically during long-running bytecode execution (based on an instruction-count or time-based interval) and explicitly around blocking I/O system calls (file reads, network sockets, `time.sleep`). For CPU-bound work, threads are constantly competing to reacquire the GIL to keep computing, adding context-switching overhead with NO parallel speedup (since only one thread can hold the GIL and run Python bytecode at any instant). For I/O-bound work, threads release the GIL WHILE WAITING on I/O, letting other threads make progress during that idle time -- true overlap, not just interleaving.

```
import time, threading

def io_task():
    time.sleep(1)  # GIL released during sleep -- other threads run freely

start = time.time()
threads = [threading.Thread(target=io_task) for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()
print(f"I/O-bound (5 threads): {time.time() - start:.2f}s")  # ~1s, not 5s

def cpu_task():
    x = 0
    for _ in range(20_000_000):
        x += 1

start = time.time()
threads = [threading.Thread(target=cpu_task) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(f"CPU-bound (4 threads): {time.time() - start:.2f}s")  # roughly same as sequential
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The GIL is switched based on '`sys.setswitchinterval()`' (default ~5ms) for pure bytecode execution, meaning CPU-bound threads DO interleave, but never truly run simultaneously.
2. Edge case: some C-extension libraries (NumPy, certain I/O and compression libraries) explicitly release the GIL during their internal C-level computation, allowing genuine parallel speedup for those specific operations even in threads.
3. Common follow-up: 'does this mean threading is useless for CPU-bound work?' -- essentially yes for pure-Python CPU-bound code; use multiprocessing or GIL-releasing C-extensions (like NumPy vectorized operations) instead.
4. Pitfall: benchmarking threaded CPU-bound code on a small workload might show a slight speedup or no clear slowdown due to noise/overhead patterns -- always test with a realistic, sufficiently large CPU-bound workload to see the GIL's true effect.
5. Edge case: excessive GIL contention across many threads can actually make CPU-bound threaded code SLOWER than single-threaded due to context-switching and lock-acquisition overhead.
6. Related: PEP 703 (accepted for Python 3.13+ as an experimental 'free-threaded' build option) aims to make the GIL optional, which would fundamentally change this tradeoff in future Python versions.

Q42. Weak References

HARD

Question: What are weak references, and when would you use them?

A weak reference (via the `weakref` module) refers to an object WITHOUT increasing its reference count, meaning the object can still be garbage collected even while a weak reference to it exists. This is useful for caches, observer/listener patterns, and parent-child object relationships where you want to avoid creating reference cycles or unintentionally keeping objects alive forever.

```
import weakref

class Cache:
    pass

obj = Cache()
weak_ref = weakref.ref(obj)

print(weak_ref())  # <Cache object> -- still alive, strong ref 'obj' exists

del obj
print(weak_ref())  # None -- the object was garbage collected

# WeakValueDictionary: entries disappear automatically when values are gc'd
cache = weakref.WeakValueDictionary()
temp = Cache()
cache["key"] = temp
print("key" in cache)  # True
del temp
print("key" in cache)  # False -- automatically removed
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Not all objects support weak references by default -- built-in types like `int`, `str`, and `list` don't support `weakref.ref()` directly without a `__weakref__` slot (custom classes get this automatically unless `__slots__` excludes it).
2. Edge case: calling a weak reference (`weak_ref()`) returns `None` if the referent has been garbage collected, or the actual object if it's still alive -- always check for `None` before using the result.
3. Common follow-up: 'how does this relate to caches?' -- `weakref.WeakValueDictionary` and `WeakKeyDictionary` automatically evict entries when their key/value objects are no longer referenced elsewhere, preventing the cache itself from being the reason objects never get freed.
4. Pitfall: using `__slots__` on a class without including `'__weakref__'` in the slots list disables weak reference support for that class entirely, potentially breaking code that expects to weakly reference instances.
5. Edge case: `weakref.proxy()` creates a proxy object that behaves like the original object (attribute access forwards through) but still becomes invalid/raises `ReferenceError` once the referent is collected.
6. Related: this is commonly used in GUI frameworks and observer patterns to let a 'subject' hold weak references to its 'observers', so observers can be garbage collected normally without the subject accidentally keeping them alive forever.

Q43. `__slots__`

HARD

Question: What does `__slots__` do, and what are its tradeoffs?

`__slots__` is a class-level declaration listing the exact set of allowed instance attributes, which tells Python to store instance data in a fixed-size, more memory-efficient structure instead of the default per-instance `__dict__`. This significantly reduces memory usage for classes with MANY instances, at the cost of losing the flexibility to dynamically add new attributes at runtime (and some complications with multiple inheritance and weak references).

```
class RegularPoint:
    def __init__(self, x, y):
        self.x, self.y = x, y

class SlottedPoint:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

import sys
r = RegularPoint(1, 2)
s = SlottedPoint(1, 2)
print(sys.getsizeof(r.__dict__)) # has a __dict__, extra overhead per instance
# s.__dict__ # AttributeError -- no __dict__ at all with __slots__

r.z = 99 # OK, dynamic attribute allowed
# s.z = 99 # AttributeError -- 'z' not declared in __slots__
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The memory savings from `__slots__` become significant when creating thousands or millions of instances (e.g. in data-processing pipelines, game entities, or graph nodes) -- for a handful of instances, the difference is negligible.
2. Edge case: subclassing a slotted class WITHOUT also defining `__slots__` in the subclass re-introduces a `__dict__` for that subclass's instances, silently negating the memory benefit.
3. Common follow-up: 'can you still use `@property` with `__slots__`?' -- yes, properties are class-level descriptors and don't require instance `__dict__` storage, so they work fine alongside slots.
4. Pitfall: forgetting to include `'__weakref__'` in `__slots__` if the class needs to support `weakref.ref()` -- by default, slotted classes don't support weak references unless explicitly declared.
5. Edge case: multiple inheritance with multiple parent classes that each define non-empty, overlapping `__slots__` raises a `TypeError` due to conflicting memory layouts -- this is a real practical limitation.
6. Related: this same memory-layout tradeoff is why libraries like `attrs` and `dataclasses(slots=True)` (Python 3.10+) offer built-in options to auto-generate slotted classes for performance-sensitive code.

Q44. Python's Import System

HARD

Question: How does Python's import statement and module caching work internally?

When you 'import module_name', Python first checks sys.modules (a cache of already-imported modules) -- if found, it reuses the cached module object instantly without re-executing the module's code. If not cached, Python searches sys.path for the module (built-ins, then the current directory/script location, then installed site-packages), executes the module's code top-to-bottom exactly once, and stores the resulting module object in sys.modules before returning it.

```
import sys

# First import: executes the module's top-level code
import json
print('json' in sys.modules)  # True

# Second import anywhere in the program: instant, reuses cache, no re-execution
import json as json2
print(json is json2)         # True -- literally the same module object

# Force re-execution (rarely needed, mainly for interactive dev/hot-reloading):
import importlib
importlib.reload(json)
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Because modules are only executed ONCE and cached, module-level code with side effects (like opening a database connection or printing a startup message) runs exactly once per process, regardless of how many times/places it's imported.
2. Edge case: circular imports (module A imports module B, which imports module A) can cause AttributeError or partial-module issues if names are accessed before the first module's execution completes -- restructuring imports or moving them inside functions often resolves this.
3. Common follow-up: 'what's the difference between import module and from module import name?' -- both trigger the same module-loading/caching mechanism; 'from...import' additionally copies a specific name into the current namespace at import time (a snapshot, not a live link).
4. Pitfall: 'from module import name' can cause confusion if the original module's 'name' is later reassigned -- the importing code's copy does NOT reflect that later change, unlike accessing module.name each time.
5. Edge case: sys.path determines where Python looks for modules, in order -- manipulating it at runtime (sys.path.insert / sys.path.append) is a common (if somewhat fragile) way to add custom import locations.
6. Related: the importlib module exposes the underlying import machinery programmatically, useful for implementing plugin systems, dynamic imports, or custom import hooks (via importlib.abc).

Q45. pickle vs marshal

HARD

Question: What's the difference between Python's pickle and marshal modules for serialization?

pickle serializes arbitrary Python objects (including custom classes, with support for versioning, custom `__reduce__`/`__getstate__` hooks, and cross-version compatibility considerations) into a byte stream that can be restored later. marshal is a much simpler, lower-level, and Python-VERSION-SPECIFIC format used internally by CPython mainly to serialize compiled bytecode (.pyc files) -- it is NOT intended for general-purpose data persistence and has no guarantee of compatibility across Python versions.

```
import pickle

data = {"name": "Alice", "scores": [90, 85, 88]}
serialized = pickle.dumps(data)
restored = pickle.loads(serialized)
print(restored == data)  # True

class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

p = Point(1, 2)
p_bytes = pickle.dumps(p)
p2 = pickle.loads(p_bytes)
print(p2.x, p2.y)  # 1 2 -- custom object round-tripped

# marshal is used internally for .pyc bytecode caching, not recommended for app data
import marshal
m = marshal.dumps([1, 2, 3])
print(marshal.loads(m))  # [1, 2, 3]
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. `pickle.load()` on untrusted data is a SERIOUS security risk -- a maliciously crafted pickle stream can execute arbitrary code during deserialization (via `__reduce__`), so never unpickle data from an untrusted source.
2. Edge case: not all objects are picklable by default -- open file handles, database connections, lambdas, and locks typically raise `TypeError/PicklingError` unless the class defines custom `__getstate__`/`__setstate__` or `__reduce__` methods.
3. Common follow-up: 'why not always use pickle for everything?' -- for cross-language interoperability or long-term/cross-version data storage, JSON or a schema-based format (Protocol Buffers, Avro) is safer and more portable than pickle's Python-specific, version-sensitive format.
4. Pitfall: pickling a class instance, then loading it in a context where that class's DEFINITION has changed or is unavailable, can silently produce a broken object or raise an import error at unpickling time.
5. Edge case: pickle has multiple protocol versions (0-5); using a newer protocol (`pickle.HIGHEST_PROTOCOL`) is more efficient but may not be readable by older Python versions expecting an older protocol.
6. Related: the `json` module is the standard, safe, cross-language-compatible alternative for simple data structures (dicts, lists, strings, numbers), though it can't natively serialize arbitrary custom objects without custom encoder/decoder logic.

Q46. Closures and Late Binding

HARD

Question: Explain Python closures, and the classic late-binding bug when creating closures inside a loop.

A closure is a nested function that 'remembers' and can access variables from its enclosing scope, even after the outer function has finished executing. Python closures use LATE BINDING -- they look up the enclosing variable's value at the time the closure is CALLED, not at the time it was CREATED. This causes a very common bug when creating multiple closures in a loop: they all end up referencing the SAME final value of the loop variable.

```
def make_multiplier(factor):
    def multiply(x):
        return x * factor    # closes over 'factor'
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
print(double(5), triple(5))    # 10 15 -- each closure has its own 'factor'

# Classic late-binding bug:
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])    # [2, 2, 2] -- NOT [0, 1, 2] as many expect!

# Fix using a default argument to capture the value at creation time:
funcs_fixed = [lambda i=i: i for i in range(3)]
print([f() for f in funcs_fixed])    # [0, 1, 2] -- correct
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The bug occurs because all the lambdas share the SAME enclosing scope's 'i' variable -- by the time any lambda is actually called, the loop has finished and 'i' holds its final value (2 in this example).
2. Edge case: this bug applies equally to regular nested 'def' functions created in a loop, not just lambdas -- the late-binding behavior is a property of closures in general, not specific to lambda syntax.
3. Common follow-up: 'what's another way to fix this besides a default argument?' -- wrapping the closure creation in an outer function call (like make_multiplier above) that takes the loop variable as a parameter, creating a fresh scope per iteration.
4. Pitfall: this bug is especially sneaky in GUI event-handler registration loops (e.g. binding a callback per button in a loop) -- all buttons end up triggering the same behavior tied to the last loop value.
5. Edge case: closures only capture variables that are actually referenced inside the nested function -- unrelated outer-scope variables aren't captured or retained, so memory isn't wasted on the entire outer scope.
6. Related: `functools.partial(func, i)` is another common technique to 'bake in' a value at creation time rather than relying on late-binding lookup, offering a slightly more explicit alternative to the default-argument trick.

Q47. Process-based vs Thread-based Parallelism for CPU-bound Work

HARD

Question: For a CPU-bound task, walk through why splitting work across processes achieves real speedup while threads don't, and what overhead processes introduce in exchange.

Each process gets its own independent Python interpreter, memory space, and GIL, so N processes can genuinely execute Python bytecode simultaneously across N CPU cores -- true parallelism. The tradeoff is overhead: spawning a process is much heavier than spawning a thread (more memory, slower startup), and since processes don't share memory, moving data between them requires explicit serialization (pickling) through IPC mechanisms like pipes, queues, or shared memory -- this overhead can outweigh the benefit for small tasks or tasks requiring frequent inter-task communication.

```
from multiprocessing import Pool
import time

def cpu_heavy(n):
    total = 0
    for i in range(n):
        total += i * i
    return total

if __name__ == "__main__":
    inputs = [10_000_000] * 4

    start = time.time()
    results_serial = [cpu_heavy(n) for n in inputs]
    print(f"Serial: {time.time() - start:.2f}s")

    start = time.time()
    with Pool(4) as pool:
        results_parallel = pool.map(cpu_heavy, inputs)
    print(f"Parallel (4 processes): {time.time() - start:.2f}s")
    # Parallel version is meaningfully faster on a multi-core machine
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The speedup from multiprocessing scales with the amount of independent, embarrassingly-parallel CPU work -- tasks that require constant back-and-forth communication between processes see much of that theoretical speedup eaten by IPC/serialization overhead.
2. Edge case: for very small or fast tasks, the overhead of pickling arguments, spawning workers, and unpickling results can make multiprocessing SLOWER than simply running the task serially -- always benchmark rather than assuming.
3. Common follow-up: 'what's the difference between fork and spawn process-start methods?' -- fork (default on Linux) duplicates the parent process's memory (faster, but can be unsafe with threads/locks); spawn (default on Windows/macOS since 3.8) starts a fresh interpreter and re-imports the module (safer, but slower to start).
4. Pitfall: assuming shared global state (like a module-level list) is visible across processes is a very common bug -- each process gets its OWN independent copy of memory, so mutations in one process are invisible to others without explicit IPC.
5. Edge case: multiprocessing.shared_memory (Python 3.8+) allows sharing raw memory buffers between processes without full serialization overhead, useful for large numeric arrays (often paired with NumPy).
6. Related: joblib and concurrent.futures.ProcessPoolExecutor provide higher-level, more ergonomic APIs over the same underlying multiprocessing mechanisms for common parallel-computation patterns.

Q48. Exception Chaining (raise ... from ...)

HARD

Question: What does 'raise NewException() from original_exception' do, and why is it useful?

Exception chaining explicitly links a newly raised exception to the exception that caused it, preserving both tracebacks so the full causal chain is visible in the output ('The above exception was the direct cause of the following exception'). Without explicit chaining, Python still shows an implicit chain when an exception is raised INSIDE an except block (as '...during handling of the above exception, another exception occurred'), but 'raise ... from ...' makes the causal relationship explicit and intentional, and 'raise ... from None' explicitly suppresses the chain when it would be noise.

```
def parse_config(raw):
    try:
        return int(raw)
    except ValueError as e:
        raise ValueError(f"Invalid config value: {raw!r}") from e

try:
    parse_config("not_a_number")
except ValueError as e:
    print(e)                # Invalid config value: 'not_a_number'
    print(e.__cause__)       # invalid literal for int() with base 10: 'not_a_number'

# Suppressing the chain entirely when the original exception is irrelevant noise:
def strict_parse(raw):
    try:
        return int(raw)
    except ValueError:
        raise ValueError("Bad input") from None  # hides the original traceback
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. The chained exception is accessible programmatically via the new exception's `__cause__` attribute (for explicit 'from') or `__context__` attribute (for implicit chaining without 'from').
2. Edge case: 'raise NewException() from None' explicitly sets `__suppress_context__` to True, cleanly hiding an implementation-detail original exception that would otherwise confuse users with irrelevant internal details.
3. Common follow-up: 'what's the difference between `__cause__` and `__context__`?' -- `__cause__` is set only with explicit 'from', representing an intentional causal link; `__context__` is set automatically any time an exception occurs while already handling another one, even without 'from'.
4. Pitfall: silently swallowing the original exception (catching it and raising a generic new one WITHOUT chaining) destroys valuable debugging information -- always chain unless there's a specific reason (like security, hiding internals) to suppress it.
5. Edge case: re-raising the SAME exception object with a bare 'raise' (no arguments) inside an except block preserves the original traceback perfectly, which is different from 'raise e' (which can subtly reset traceback info in some contexts).
6. Related: this feature (PEP 3134) is especially valuable in layered systems (web frameworks, ORMs) where a low-level error (e.g. a database connection failure) needs to be wrapped in a higher-level, more meaningful application exception without losing the root cause.

Q49. contextvars vs threading.local

HARD

Question: What are context variables (contextvars), and how do they differ from threading.local for managing per-context state?

threading.local() provides storage that's isolated PER THREAD -- each thread sees its own independent value. contextvars.ContextVar (added in Python 3.7) provides storage isolated PER LOGICAL CONTEXT, which correctly follows async/await task boundaries and generator execution -- critical because asyncio runs many concurrent 'tasks' on a SINGLE thread, so threading.local can't distinguish between them, while ContextVar can.

```
import contextvars
import asyncio

request_id = contextvars.ContextVar("request_id", default=None)

async def handle_request(rid):
    request_id.set(rid)
    await asyncio.sleep(0.1)
    print(f"Handling request: {request_id.get()}")

async def main():
    # Both run "concurrently" on the SAME thread via asyncio
    await asyncio.gather(handle_request("req-1"), handle_request("req-2"))
    # Each correctly prints its OWN request_id, not a shared/overwritten one

asyncio.run(main())
# Handling request: req-1
# Handling request: req-2
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. threading.local() would be USELESS for the above example, since both coroutines run on the exact same OS thread -- they'd incorrectly share (and overwrite) a single threading.local value.
2. Edge case: ContextVar values are automatically copied into each new asyncio Task when it's created, so modifications made inside one task don't leak into sibling tasks, even though they share a thread and event loop.
3. Common follow-up: 'what's a real use case for ContextVar?' -- request-scoped data in async web frameworks (like a request ID, current user, or trace ID for logging/tracing), where you need per-logical-request isolation without threading.
4. Pitfall: using a plain global variable (or threading.local) to track 'current user' or 'request ID' in an async web application will cause cross-request data leakage/corruption under concurrent load -- ContextVar is the correct tool.
5. Edge case: ContextVar.set() returns a Token object that can be passed to ContextVar.reset(token) to explicitly restore the previous value, useful for scoped/temporary overrides.
6. Related: contextvars.copy_context() lets you snapshot and run a callable within a captured context, which is exactly the mechanism asyncio uses internally to propagate context correctly across scheduled tasks.

Q50. Small Integer / String Interning and 'is'

HARD

Question: Explain how CPython's small-integer and string interning caching affects the behavior of the 'is' operator, and why relying on it is dangerous.

CPython pre-allocates and caches small integers in the range -5 to 256 at interpreter startup, so any reference to, say, the integer 100 anywhere in your program reuses the SAME cached object -- making 'is' comparisons on these values misleadingly appear to work like '=='. Similarly, CPython interns certain strings (short identifier-like strings, and string literals that look like valid Python identifiers) so identical string literals in the same compiled code unit may also share the same object. Neither behavior is guaranteed by the Python language specification -- it's a CPython-specific implementation detail that can differ between Python versions or implementations (PyPy, Jython).

```
a = 100
b = 100
print(a is b)           # True -- both point to the same cached small-int object

x = 1000
y = 1000
print(x is y)           # False in most cases -- outside the small-int cache range
                        # (though it CAN be True in some contexts, like same code object!)

s1 = "hello"
s2 = "hello"
print(s1 is s2)         # often True -- string literal interning

s3 = "hello world!"
s4 = "hello world!"
print(s3 is s4)         # implementation-detail dependent, don't rely on it

import sys
print(sys.intern("hello world!") is sys.intern("hello world!")) # True -- explicit interning
```

6 Key Points (Edge Cases, Pitfalls & Follow-Ups)

1. Never use 'is' to compare numbers or strings for equality in real code -- always use '==', since interning behavior is an unreliable, unspecified implementation detail that can silently change between Python versions or implementations.
2. Edge case: two variables assigned the same integer or string literal WITHIN THE SAME compiled code block (function/module) are more likely to be interned/cached together than the same values computed separately at runtime (e.g. via string concatenation or arithmetic).
3. Common follow-up: 'why does CPython do this at all?' -- it's a memory and performance optimization, since small integers and common strings (like variable/attribute names) are used extremely frequently throughout typical programs.
4. Pitfall: writing test code or tutorials that rely on 'is' equality for small integers can pass by coincidence in CPython but silently produce different (and correct-per-spec) results on PyPy or in future CPython versions.
5. Edge case: `sys.intern()` lets you EXPLICITLY force string interning for performance-critical code doing many repeated string comparisons/dictionary lookups, making the optimization deliberate and correct rather than accidental.
6. Related: this same category of 'don't rely on implementation details' extends to relying on CPython's dict insertion-order guarantee behaving identically across all Python implementations before it became an official language guarantee in 3.7.